

ORM Validation and Properties : Code-Along (CodeGrade)

Learning Goals

- Evolve attributes to properties.
 - Adapt ORM methods to validate object state with properties
-

Key Vocab

- **Object-Relational Mapping (ORM)**: a programming technique that provides a mapping between an object-oriented data model and a relational database model.
 - **Attribute**: variables that belong to an object.
 - **Property**: attributes that are controlled by methods.
 - **Decorator**: function that takes another function as an argument and returns a new function with added functionality.
-

Introduction

Let's update our company data model to add some constraints on the `Department` and `Employee` attributes:

- The `Department` name and location must be non-empty strings.
- The `Employee` name and job title must be non-empty strings.
- The `Employee` department_id must be a foreign key reference to a `Department` object that has been persisted to the database.

We'll evolve the attributes to be managed by property methods, with setter methods that ensure the attributes are assigned valid values.

Code Along

This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
pipenv install
pipenv shell
```

Evolve `Department` attributes to properties

We'll start by evolving the `Department` class to add property methods to manage the `name` and `location` attributes, as shown in the code below. The setter methods will check for non-empty string values prior to updating the object state:

```
from __init__ import CURSOR, CONN

class Department:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, location, id=None):
        self.id = id
        self.name = name
        self.location = location

    def __repr__(self):
        return f"<Department {self.id}: {self.name}, {self.location}>"
```

```
@property
def name(self):
    return self._name

@name.setter
def name(self, name):
    if isinstance(name, str) and len(name):
        self._name = name
    else:
        raise ValueError(
            "Name must be a non-empty string"
        )

@property
def location(self):
    return self._location

@location.setter
def location(self, location):
    if isinstance(location, str) and len(location):
        self._location = location
    else:
        raise ValueError(
            "Location must be a non-empty string"
        )

# Existing ORM methods ....
```

We'll also update the `Employee` class to add property methods to manage the `name`, `job_title` and `department_id` attributes. Note the `department_id` setter method checks to ensure we are assigning a valid department by checking the foreign key reference in the database:

```
from __init__ import CURSOR, CONN
from department import Department

class Employee:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, job_title, department_id, id=None):
        self.id = id
        self.name = name
        self.job_title = job_title
        self.department_id = department_id

    def __repr__(self):
        return (
            f"<Employee {self.id}: {self.name}, {self.job_title}, "
            + f"Department: {self.department.name} >"
        )

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, name):
        if isinstance(name, str) and len(name):
            self._name = name
        else:
            raise ValueError(
                "Name must be a non-empty string"
            )
```

```
)

@property
def job_title(self):
    return self._job_title

@job_title.setter
def job_title(self, job_title):
    if isinstance(job_title, str) and len(job_title):
        self._job_title = job_title
    else:
        raise ValueError(
            "job_title must be a non-empty string"
        )

@property
def department_id(self):
    return self._department_id

@department_id.setter
def department_id(self, department_id):
    if type(department_id) is int and Department.find_by_id(department_id):
        self._department_id = department_id
    else:
        raise ValueError(
            "department_id must reference a department in the database")

# Existing ORM methods ....
```

The `lib/testing` folder has two new test files for testing the properties, `department_property_test.py` and `employee_property_test.py` . Run the tests to ensure the property validations work correctly:

```
pytest -x
```

You can also use an `ipdb` session to test the properties.

```
python lib/debug.py
```

Type each statement one at a time into the `ipdb>` prompt:

```
ipdb> Department.get_all()
[<Department 1: Payroll, Building A, 5th Floor>, <Department 2: Human Resources, Building C, East Wing>]
ipdb> payroll = Department.find_by_name("Payroll")
ipdb> payroll
<Department 1: Payroll, Building A, 5th Floor>
ipdb> payroll.location = 7
*** ValueError: Location must be a non-empty string
ipdb>
```

Let's try to set an invalid department id for an employee:

```
ipdb> employee = Employee.find_by_id(1)
ipdb> employee
<Employee 1: Amir, Accountant, Department ID: 1>
ipdb> employee.department_id = 1000
*** ValueError: department_id must reference a department in the database
```

Conclusion

Properties manage the access and mutation of attributes. We can use property setter methods to ensure we assign valid values prior to persisting objects to the database.

Solution Code

```
# lib/department.py
from __init__ import CURSOR, CONN

class Department:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, location, id=None):
        self.id = id
        self.name = name
        self.location = location

    def __repr__(self):
        return f"<Department {self.id}: {self.name}, {self.location}>"

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, name):
        if isinstance(name, str) and len(name):
            self._name = name
        else:
            raise ValueError(
                "Name must be a non-empty string"
            )

    @property
    def location(self):
```

```
        return self._location

    @location.setter
    def location(self, location):
        if isinstance(location, str) and len(location):
            self._location = location
        else:
            raise ValueError(
                "Location must be a non-empty string"
            )

    @classmethod
    def create_table(cls):
        """ Create a new table to persist the attributes of Department instances """
        sql = """
            CREATE TABLE IF NOT EXISTS departments (
                id INTEGER PRIMARY KEY,
                name TEXT,
                location TEXT)
        """
        CURSOR.execute(sql)
        CONN.commit()

    @classmethod
    def drop_table(cls):
        """ Drop the table that persists Department instances """
        sql = """
            DROP TABLE IF EXISTS departments;
        """
        CURSOR.execute(sql)
        CONN.commit()

    def save(self):
```



```
""" Insert a new row with the name and location values of the current Department instance.
Update object id attribute using the primary key value of new row.
Save the object in local dictionary using table row's PK as dictionary key"""
```

```
sql = """
    INSERT INTO departments (name, location)
    VALUES (?, ?)
    """

CURSOR.execute(sql, (self.name, self.location))
CONN.commit()

self.id = CURSOR.lastrowid
type(self).all[self.id] = self
```

```
@classmethod
```

```
def create(cls, name, location):
    """ Initialize a new Department instance and save the object to the database """
    department = cls(name, location)
    department.save()
    return department
```

```
def update(self):
    """Update the table row corresponding to the current Department instance."""
    sql = """
        UPDATE departments
        SET name = ?, location = ?
        WHERE id = ?
        """

    CURSOR.execute(sql, (self.name, self.location, self.id))
    CONN.commit()
```

```
def delete(self):
    """Delete the table row corresponding to the current Department instance,
```

delete the dictionary entry, and reassign id attribute"""

```
sql = """
    DELETE FROM departments
    WHERE id = ?
    """
```

```
CURSOR.execute(sql, (self.id,))
CONN.commit()
```

Delete the dictionary entry using id as the key

```
del type(self).all[self.id]
```

Set the id to None

```
self.id = None
```

@classmethod

```
def instance_from_db(cls, row):
```

"""Return a Department object having the attribute values from the table row."""

Check the dictionary for an existing instance using the row's primary key

```
department = cls.all.get(row[0])
```

```
if department:
```

ensure attributes match row values in case local instance was modified

```
department.name = row[1]
```

```
department.location = row[2]
```

```
else:
```

not in dictionary, create new instance and add to dictionary

```
department = cls(row[1], row[2])
```

```
department.id = row[0]
```

```
cls.all[department.id] = department
```

```
return department
```

```
@classmethod
def get_all(cls):
    """Return a list containing a Department object per row in the table"""
    sql = """
        SELECT *
        FROM departments
    """

    rows = CURSOR.execute(sql).fetchall()

    return [cls.instance_from_db(row) for row in rows]

@classmethod
def find_by_id(cls, id):
    """Return a Department object corresponding to the table row matching the specified primary key"""
    sql = """
        SELECT *
        FROM departments
        WHERE id = ?
    """

    row = CURSOR.execute(sql, (id,)).fetchone()
    return cls.instance_from_db(row) if row else None

@classmethod
def find_by_name(cls, name):
    """Return a Department object corresponding to first table row matching specified name"""
    sql = """
        SELECT *
        FROM departments
        WHERE name is ?
    """
```

```
row = CURSOR.execute(sql, (name,)).fetchone()
return cls.instance_from_db(row) if row else None
```

```
def employees(self):
    """Return list of employees associated with current department"""
    from employee import Employee
    sql = """
        SELECT * FROM employees
        WHERE department_id = ?
    """
    CURSOR.execute(sql, (self.id,))

    rows = CURSOR.fetchall()
    return [
        Employee.instance_from_db(row) for row in rows
    ]
```

```
# lib/employee.py
from __init__ import CURSOR, CONN
from department import Department
```

```
class Employee:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, job_title, department_id, id=None):
        self.id = id
        self.name = name
        self.job_title = job_title
        self.department_id = department_id

    def __repr__(self):
```

```
    return (  
        f"<Employee {self.id}: {self.name}, {self.job_title}, " +  
        f"Department ID: {self.department_id}>"  
    )
```

```
@property  
def name(self):  
    return self._name
```

```
@name.setter  
def name(self, name):  
    if isinstance(name, str) and len(name):  
        self._name = name  
    else:  
        raise ValueError(  
            "Name must be a non-empty string"  
        )
```

```
@property  
def job_title(self):  
    return self._job_title
```

```
@job_title.setter  
def job_title(self, job_title):  
    if isinstance(job_title, str) and len(job_title):  
        self._job_title = job_title  
    else:  
        raise ValueError(  
            "job_title must be a non-empty string"  
        )
```

```
@property  
def department_id(self):
```

```
    return self._department_id

@department_id.setter
def department_id(self, department_id):
    if type(department_id) is int and Department.find_by_id(department_id):
        self._department_id = department_id
    else:
        raise ValueError(
            "department_id must reference a department in the database")

@classmethod
def create_table(cls):
    """ Create a new table to persist the attributes of Employee instances """
    sql = """
        CREATE TABLE IF NOT EXISTS employees (
            id INTEGER PRIMARY KEY,
            name TEXT,
            job_title TEXT,
            department_id INTEGER,
            FOREIGN KEY (department_id) REFERENCES departments(id))
    """
    CURSOR.execute(sql)
    CONN.commit()

@classmethod
def drop_table(cls):
    """ Drop the table that persists Employee instances """
    sql = """
        DROP TABLE IF EXISTS employees;
    """
    CURSOR.execute(sql)
    CONN.commit()
```

```
def save(self):
    """ Insert a new row with the name, job title, and department id values of the current Employee object.
    Update object id attribute using the primary key value of new row.
    Save the object in local dictionary using table row's PK as dictionary key"""
    sql = """
        INSERT INTO employees (name, job_title, department_id)
        VALUES (?, ?, ?)
    """

    CURSOR.execute(sql, (self.name, self.job_title, self.department_id))
    CONN.commit()

    self.id = CURSOR.lastrowid
    type(self).all[self.id] = self

def update(self):
    """Update the table row corresponding to the current Employee instance."""
    sql = """
        UPDATE employees
        SET name = ?, job_title = ?, department_id = ?
        WHERE id = ?
    """
    CURSOR.execute(sql, (self.name, self.job_title,
                        self.department_id, self.id))
    CONN.commit()

def delete(self):
    """Delete the table row corresponding to the current Employee instance,
    delete the dictionary entry, and reassign id attribute"""

    sql = """
        DELETE FROM employees
        WHERE id = ?
    """
```

```
"""
```

```
CURSOR.execute(sql, (self.id,))  
CONN.commit()
```

```
# Delete the dictionary entry using id as the key  
del type(self).all[self.id]
```

```
# Set the id to None  
self.id = None
```

```
@classmethod
```

```
def create(cls, name, job_title, department_id):  
    """ Initialize a new Employee instance and save the object to the database """  
    employee = cls(name, job_title, department_id)  
    employee.save()  
    return employee
```

```
@classmethod
```

```
def instance_from_db(cls, row):  
    """Return an Employee object having the attribute values from the table row."""  
  
    # Check the dictionary for existing instance using the row's primary key  
    employee = cls.all.get(row[0])  
    if employee:  
        # ensure attributes match row values in case local instance was modified  
        employee.name = row[1]  
        employee.job_title = row[2]  
        employee.department_id = row[3]  
    else:  
        # not in dictionary, create new instance and add to dictionary  
        employee = cls(row[1], row[2], row[3])  
        employee.id = row[0]
```



```
        cls.all[employee.id] = employee
    return employee
```

```
@classmethod
```

```
def get_all(cls):
```

```
    """Return a list containing one Employee object per table row"""
```

```
    sql = """
```

```
        SELECT *
```

```
        FROM employees
```

```
    """
```

```
    rows = CURSOR.execute(sql).fetchall()
```

```
    return [cls.instance_from_db(row) for row in rows]
```

```
@classmethod
```

```
def find_by_id(cls, id):
```

```
    """Return Employee object corresponding to the table row matching the specified primary key"""
```

```
    sql = """
```

```
        SELECT *
```

```
        FROM employees
```

```
        WHERE id = ?
```

```
    """
```

```
    row = CURSOR.execute(sql, (id,)).fetchone()
```

```
    return cls.instance_from_db(row) if row else None
```

```
@classmethod
```

```
def find_by_name(cls, name):
```

```
    """Return Employee object corresponding to first table row matching specified name"""
```

```
    sql = """
```

```
        SELECT *
```

```
        FROM employees
```

```
        WHERE name is ?
    """

    row = CURSOR.execute(sql, (name,)).fetchone()
    return cls.instance_from_db(row) if row else None
```